And here is the output of the program.

```
Inside fun1
Inside main
Inside fun2
```

Note that the functions **fun1( )** and **fun2( )** should neither receive nor return any value. If we want two functions to get executed at startup then their pragmas should be defined in the reverse order in which you want to get them called.

(b) **#pragma warn:** This directive tells the compiler whether or not we want to suppress a specific warning. Usage of this pragma is shown below.

```
#pragma warn –rvl    /* return value */
#pragma warn –par    /* parameter not used */
#pragma warn –rch    /* unreachable code */

int  f1( )
{
    int  a = 5 ;
}

void  f2 ( int  x )
{
    printf ( "\nInside f2" ) ;
}

int  f3( )
{
    int  x = 6 ;
    return x ;
    x++  ;
}

void  main( )
```

```
{
    f1( ) ;
    f2 ( 7 ) ;
    f3( ) ;
}
```

If you go through the program you can notice three problems immediately. These are:

(a) Though promised, **f1( )** doesn't return a value.

(b) The parameter **x** that is passed to **f2( )** is not being used anywhere in **f2( )**.

(c) The control can never reach **x++** in **f3( )**.

If we compile the program we should expect warnings indicating the above problems. However, this does not happen since we have suppressed the warnings using the **#pragma** directives. If we replace the '–' sign with a '+' then these warnings would be flashed on compilation. Though it is a bad practice to suppress warnings, at times it becomes useful to suppress them. For example, if you have written a huge program and are trying to compile it, then to begin with you are more interested in locating the errors, rather than the warnings. At such times you may suppress the warnings. Once you have located all errors, then you may turn on the warnings and sort them out.

Summary

(a) The preprocessor directives enable the programmer to write programs that are easy to develop, read, modify and transport to a different computer system.

(b) We can make use of various preprocessor directives such as **#define**, **#include**, **#ifdef** - **#else** - **#endif**, **#if** and **#elif** in our program.

(c) The directives like **#undef** and **#pragma** are also useful although they are seldom used.

Exercise

[A] Answer the following:

(a) What is a preprocessor directive

1. a message from compiler to the programmer
2. a message from compiler to the linker
3. a message from programmer to the preprocessor
4. a message from programmer to the microprocessor

(b) Which of the following are correctly formed **#define** statements:

```
#define   INCH PER FEET  12
#define   SQR (X)  ( X * X )
#define   SQR(X)   X * X
#define   SQR(X)  ( X * X )
```

(c) State True or False:

1. A macro must always be written in capital letters.
2. A macro should always be accomodated in a single line.
3. After preprocessing when the program is sent for compilation the macros are removed from the expanded source code.
4. Macros with arguments are not allowed.
5. Nested macros are allowed.
6. In a macro call the control is passed to the macro.

(d) How many **#include** directives can be there in a given program file?

(e) What is the difference between the following two **#include** directives:

```
#include  "conio.h"
#include <conio.h>
```

(f) A header file is:

1. A file that contains standard library functions
2. A file that contains definitions and macros
3. A file that contains user - defined functions
4. A file that is present in current working directory

(g) Which of the following is not a preprocessor directive

1. #if
2. #elseif
3. #undef
4. #pragma

(h) All macro substitutions in a program are done

1. Before compilation of the program
2. After compilation
3. During execution
4. None of the above

(i) In a program the statement:

```
#include "filename"
```

is replaced by the contents of the file “filename”

1. Before compilation
2. After Compilation
3. During execution
4. None of the above

[B] What would be the output of the following program:

(a)
```
main( )
{
    int  i = 2 ;
    #ifdef DEF
        i *= i ;
    #else
        printf ( "\n%d", i ) ;
    #endif
}
```

(b)
```
#define PRODUCT(x) ( x * x )
main( )
{
    int  i = 3, j ;
    j = PRODUCT( i + 1 ) ;
    printf ( "\n%d", j ) ;
}
```

(c)
```
#define PRODUCT(x) ( x * x )
main( )
{
    int  i = 3, j, k ;
    j = PRODUCT( i++ ) ;
    k = PRODUCT ( ++i ) ;

    printf ( "\n%d %d", j, k ) ;
}
```

(d)
```
# define SEMI ;
main()
{
  int p = 3 SEMI ;
  printf ( "%d", p ) SEMI
}
```

[C] Attempt the following:

(a) Write down macro definitions for the following:

1. To test whether a character entered is a small case letter or not.
2. To test whether a character entered is a upper case letter or not.
3. To test whether a character is an alphabet or not. Make use of the macros you defined in (1) and (2) above.
4. To obtain the bigger of two numbers.

(b) Write macro definitions with arguments for calculation of area and perimeter of a triangle, a square and a circle. Store these macro definitions in a file called "areaperi.h". Include this file in your program, and call the macro definitions for calculating area and perimeter for different squares, triangles and circles.

(c) Write down macro definitions for the following:

1. To find arithmetic mean of two numbers.
2. To find absolute value of a number.
3. To convert a uppercase alphabet to lowercase.
4. To obtain the bigger of two numbers.

(d) Write macro definitions with arguments for calculation of Simple Interest and Amount. Store these macro definitions in a file called "interest.h". Include this file in your program, and use the macro definitions for calculating simple interest and amount.

8 Arrays

- What are Arrays
  - A Simple Program Using Array
- More on Arrays
  - Array Initialisation
  - Bounds Checking
  - Passing Array Elements to a Function
- Pointers and Arrays
  - Passing an Entire Array to a Function
  - The Real Thing
- Two Dimensional Arrays
  - Initialising a 2-Dimensional Array
  - Memory Map of a 2-Dimensional Array
  - Pointers and 2-Dimensional Arrays
  - Pointer to an Array
  - Passing 2-D Array to a Function
- Array of Pointers
- Three-Dimensional Array
- Summary
- Exercise

The C language provides a capability that enables the user to design a set of similar data types, called array. This chapter describes how arrays can be created and manipulated in C.

We should note that, in many C books and courses arrays and pointers are taught separately. I feel it is worthwhile to deal with these topics together. This is because pointers and arrays are so closely related that discussing arrays without discussing pointers would make the discussion incomplete and wanting. In fact all arrays make use of pointers internally. Hence it is all too relevant to study them together rather than as isolated topics.

What are Arrays

For understanding the arrays properly, let us consider the following program:

```
main( )
{
    int  x ;
    x = 5 ;
    x = 10 ;
    printf ( "\nx = %d", x ) ;
}
```

No doubt, this program will print the value of **x** as 10. Why so? Because when a value 10 is assigned to **x**, the earlier value of **x**, i.e. 5, is lost. Thus, ordinary variables (the ones which we have used so far) are capable of holding only one value at a time (as in the above example). However, there are situations in which we would want to store more than one value at a time in a single variable.

For example, suppose we wish to arrange the percentage marks obtained by 100 students in ascending order. In such a case we have two options to store these marks in memory:

(a) Construct 100 variables to store percentage marks obtained by 100 different students, i.e. each variable containing one student's marks.

(b) Construct one variable (called array or subscripted variable) capable of storing or holding all the hundred values.

Obviously, the second alternative is better. A simple reason for this is, it would be much easier to handle one variable than handling 100 different variables. Moreover, there are certain logics that cannot be dealt with, without the use of an array. Now a formal definition of an array—An array is a collective name given to a group of 'similar quantities'. These similar quantities could be percentage marks of 100 students, or salaries of 300 employees, or ages of 50 employees. What is important is that the quantities must be 'similar'. Each member in the group is referred to by its position in the group. For example, assume the following group of numbers, which represent percentage marks obtained by five students.

per = { 48, 88, 34, 23, 96 }

If we want to refer to the second number of the group, the usual notation used is per_2. Similarly, the fourth number of the group is referred as per_4. However, in C, the fourth number is referred as **per[3].** This is because in C the counting of elements begins with 0 and not with 1. Thus, in this example **per[3]** refers to 23 and **per[4]** refers to 96. In general, the notation would be **per[i]**, where, **i** can take a value 0, 1, 2, 3, or 4, depending on the position of the element being referred. Here **per** is the subscripted variable (array), whereas **i** is its subscript.

Thus, an array is a collection of similar elements. These similar elements could be all **int**s, or all **float**s, or all **char**s, etc. Usually, the array of characters is called a 'string', whereas an array of **int**s or **float**s is called simply an array. Remember that all elements of

any given array must be of the same type. i.e. we cannot have an array of 10 numbers, of which 5 are **int**s and 5 are **float**s.

A Simple Program Using Array

Let us try to write a program to find average marks obtained by a class of 30 students in a test.

```
main( )
{
      int  avg, sum = 0 ;
      int  i ;
      int  marks[30] ;  /* array declaration */

      for ( i = 0 ; i <= 29 ; i++ )
      {
            printf ( "\nEnter marks " ) ;
            scanf ( "%d", &marks[i] ) ;  /* store data in array */
      }

      for ( i = 0 ; i <= 29 ; i++ )
            sum = sum + marks[i] ;  /* read data from an array*/

      avg = sum / 30 ;
      printf ( "\nAverage marks = %d", avg ) ;
}
```

There is a lot of new material in this program, so let us take it apart slowly.

Array Declaration

To begin with, like other variables an array needs to be declared so that the compiler will know what kind of an array and how large an array we want. In our program we have done this with the statement:

```
int  marks[30] ;
```

Here, **int** specifies the type of the variable, just as it does with ordinary variables and the word **marks** specifies the name of the variable. The **[30]** however is new. The number 30 tells how many elements of the type **int** will be in our array. This number is often called the 'dimension' of the array. The bracket ([ ]) tells the compiler that we are dealing with an array.

Accessing Elements of an Array

Once an array is declared, let us see how individual elements in the array can be referred. This is done with subscript, the number in the brackets following the array name. This number specifies the element's position in the array. All the array elements are numbered, starting with 0. Thus, **marks[2]** is not the second element of the array, but the third. In our program we are using the variable **i** as a subscript to refer to various elements of the array. This variable can take different values and hence can refer to the different elements in the array in turn. This ability to use variables as subscripts is what makes arrays so useful.

Entering Data into an Array

Here is the section of code that places data into an array:

```
for ( i = 0 ; i <= 29 ; i++ )
{
     printf ( "\nEnter marks " ) ;
     scanf ( "%d", &marks[i] ) ;
}
```

The **for** loop causes the process of asking for and receiving a student's marks from the user to be repeated 30 times. The first time through the loop, **i** has a value 0, so the **scanf( )** function will cause the value typed to be stored in the array element **marks[0],** the first element of the array. This process will be repeated until **i**

becomes 29. This is last time through the loop, which is a good thing, because there is no array element like **marks[30]**.

In **scanf( )** function, we have used the "address of" operator (&) on the element **marks[i]** of the array, just as we have used it earlier on other variables **(&rate**, for example). In so doing, we are passing the address of this particular array element to the **scanf( )** function, rather than its value; which is what **scanf( )** requires.

Reading Data from an Array

The balance of the program reads the data back out of the array and uses it to calculate the average. The **for** loop is much the same, but now the body of the loop causes each student's marks to be added to a running total stored in a variable called **sum**. When all the marks have been added up, the result is divided by 30, the number of students, to get the average.

```
for ( i = 0 ; i <= 29 ; i++ )
      sum = sum + marks[i] ;

avg = sum / 30 ;
printf ( "\nAverage marks = %d", avg ) ;
```

To fix our ideas, let us revise whatever we have learnt about arrays:

(a) An array is a collection of similar elements.
(b) The first element in the array is numbered 0, so the last element is 1 less than the size of the array.
(c) An array is also known as a subscripted variable.
(d) Before using an array its type and dimension must be declared.
(e) However big an array its elements are always stored in contiguous memory locations. This is a very important point which we would discuss in more detail later on.

More on Arrays

Array is a very popular data type with C programmers. This is because of the convenience with which arrays lend themselves to programming. The features which make arrays so convenient to program would be discussed below, along with the possible pitfalls in using them.

Array Initialisation

So far we have used arrays that did not have any values in them to begin with. We managed to store values in them during program execution. Let us now see how to initialize an array while declaring it. Following are a few examples that demonstrate this.

```
int  num[6] = { 2, 4, 12, 5, 45, 5 } ;
int  n[ ] = { 2, 4, 12, 5, 45, 5 } ;
float  press[ ] = { 12.3, 34.2 -23.4, -11.3 } ;
```

Note the following points carefully:

(a) Till the array elements are not given any specific values, they are supposed to contain garbage values.

(b) If the array is initialised where it is declared, mentioning the dimension of the array is optional as in the 2nd example above.

Array Elements in Memory

Consider the following array declaration:

```
int  arr[8] ;
```

What happens in memory when we make this declaration? 16 bytes get immediately reserved in memory, 2 bytes each for the 8 integers (under Windows/Linux the array would occupy 32 bytes

as each integer would occupy 4 bytes). And since the array is not being initialized, all eight values present in it would be garbage values. This so happens because the storage class of this array is assumed to be **auto**. If the storage class is declared to be **static** then all the array elements would have a default initial value as zero. Whatever be the initial values, all the array elements would always be present in contiguous memory locations. This arrangement of array elements in memory is shown in Figure 8.1.

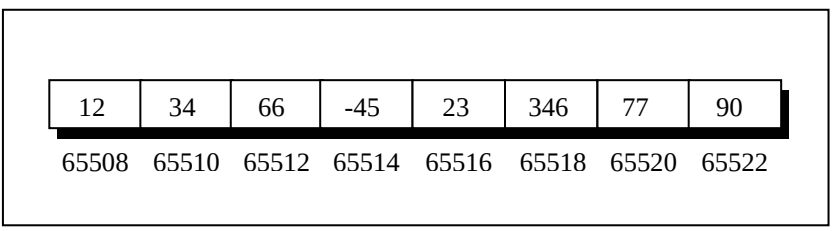


Figure 8.1

Bounds Checking

In C there is no check to see if the subscript used for an array exceeds the size of the array. Data entered with a subscript exceeding the array size will simply be placed in memory outside the array; probably on top of other data, or on the program itself. This will lead to unpredictable results, to say the least, and there will be no error message to warn you that you are going beyond the array size. In some cases the computer may just hang. Thus, the following program may turn out to be suicidal.

```
main( )
{
    int  num[40], i ;

    for ( i = 0 ; i <= 100 ; i++ )
        num[i] = i ;
}
```

Thus, to see to it that we do not reach beyond the array size is entirely the programmer's botheration and not the compiler's.

Passing Array Elements to a Function

Array elements can be passed to a function by calling the function by value, or by reference. In the call by value we pass values of array elements to the function, whereas in the call by reference we pass addresses of array elements to the function. These two calls are illustrated below:

```
/* Demonstration of call by value */
main( )
{
      int  i ;
      int  marks[ ] = { 55, 65, 75, 56, 78, 78, 90 } ;

      for ( i = 0 ; i <= 6 ; i++ )
            display ( marks[i] ) ;
}

display ( int  m )
{
      printf ( "%d ", m ) ;
}
```

And here's the output...

```
55 65 75 56 78 78 90
```

Here, we are passing an individual array element at a time to the function **display( )** and getting it printed in the function **display( ).** Note that since at a time only one element is being passed, this element is collected in an ordinary integer variable **m**, in the function **display( )**.

And now the call by reference.

```
/* Demonstration of call by reference */
main( )
{
      int  i ;
      int  marks[ ] = { 55, 65, 75, 56, 78, 78, 90 } ;

      for ( i = 0 ; i <= 6 ; i++ )
            disp ( &marks[i] ) ;
}

disp ( int  *n )
{
      printf ( "%d ", *n ) ;
}
```

And here's the output...

```
55 65 75 56 78 78 90
```

Here, we are passing addresses of individual array elements to the function **display( )**. Hence, the variable in which this address is collected (**n**) is declared as a pointer variable. And since **n** contains the address of array element, to print out the array element we are using the 'value at address' operator **(*)**.

Read the following program carefully. The purpose of the function **disp( )** is just to display the array elements on the screen. The program is only partly complete. You are required to write the function **show( )** on your own. Try your hand at it.

```
main( )
{
      int  i ;
      int  marks[ ] = { 55, 65, 75, 56, 78, 78, 90 } ;

      for ( i = 0 ; i <= 6 ; i++ )
            disp ( &marks[i] ) ;
```

```
}

disp ( int  *n )
{
     show ( &n ) ;
}
```

Pointers and Arrays

To be able to see what pointers have got to do with arrays, let us first learn some pointer arithmetic. Consider the following example:

```
main( )
{
     int  i = 3, *x ;
     float  j = 1.5, *y ;
     char  k = 'c', *z ;

     printf ( "\nValue of i = %d", i ) ;
     printf ( "\nValue of j = %f", j ) ;
     printf ( "\nValue of k = %c", k ) ;
     x = &i ;
     y = &j ;
     z = &k ;
     printf ( "\nOriginal address in x = %u", x ) ;
     printf ( "\nOriginal address in y = %u", y ) ;
     printf ( "\nOriginal address in z = %u", z ) ;
     x++ ;
     y++ ;
     z++ ;
     printf ( "\nNew address in x = %u", x ) ;
     printf ( "\nNew address in y = %u", y ) ;
     printf ( "\nNew address in z = %u", z ) ;
}
```

Here is the output of the program.

```
Value of i = 3
Value of j = 1.500000
Value of k = c
Original address in x = 65524
Original address in y = 65520
Original address in z = 65519
New address in x = 65526
New address in y = 65524
New address in z = 65520
```

Observe the last three lines of the output. 65526 is original value in **x** plus 2, 65524 is original value in **y** plus 4, and 65520 is original value in **z** plus 1. This so happens because every time a pointer is incremented it points to the immediately next location of its type. That is why, when the integer pointer **x** is incremented, it points to an address two locations after the current location, since an **int** is always 2 bytes long (under Windows/Linux since **int** is 4 bytes long, new value of **x** would be 65528). Similarly, **y** points to an address 4 locations after the current location and **z** points 1 location after the current location. This is a very important result and can be effectively used while passing the entire array to a function.

The way a pointer can be incremented, it can be decremented as well, to point to earlier locations. Thus, the following operations can be performed on a pointer:

(a) Addition of a number to a pointer. For example,

```
int  i = 4, *j, *k ;
j = &i ;
j = j + 1 ;
j = j + 9 ;
k = j + 3 ;
```

(b) Subtraction of a number from a pointer. For example,

```
int  i = 4, *j, *k ;
j = &i ;
j = j - 2 ;
j = j - 5 ;
k = j - 6 ;
```

(c) Subtraction of one pointer from another.

One pointer variable can be subtracted from another provided both variables point to elements of the same array. The resulting value indicates the number of bytes separating the corresponding array elements. This is illustrated in the following program.

```
main( )
{
    int  arr[ ] = { 10, 20, 30, 45, 67, 56, 74 } ;
    int  *i, *j ;

    i = &arr[1] ;
    j = &arr[5] ;
    printf ( "%d %d", j - i, *j - *i ) ;
}
```

Here **i** and **j** have been declared as integer pointers holding addresses of first and fifth element of the array respectively.

Suppose the array begins at location 65502, then the elements **arr[1]** and **arr[5]** would be present at locations 65504 and 65512 respectively, since each integer in the array occupies two bytes in memory. The expression **j - i** would print a value 4 and not 8. This is because **j** and **i** are pointing to locations that are 4 integers apart. What would be the result of the expression ***j - *i**? 36, since ***j** and ***i** return the values present at addresses contained in the pointers **j** and **i**.

(d) Comparison of two pointer variables

Pointer variables can be compared provided both variables point to objects of the same data type. Such comparisons can be useful when both pointer variables point to elements of the same array. The comparison can test for either equality or inequality. Moreover, a pointer variable can be compared with zero (usually expressed as NULL). The following program illustrates how the comparison is carried out.

```
main( )
{
    int  arr[ ] = { 10, 20, 36, 72, 45, 36 } ;
    int  *j, *k ;

    j = &arr [ 4 ] ;
    k = ( arr + 4 ) ;

    if ( j == k )
        printf ( "The two pointers point to the same location" ) ;
    else
        printf ( "The two pointers do not point to the same location" ) ;
}
```

A word of caution! Do not attempt the following operations on pointers... they would never work out.

(a) Addition of two pointers
(b) Multiplication of a pointer with a constant
(c) Division of a pointer with a constant

Now we will try to correlate the following two facts, which we have learnt above:

(a) Array elements are always stored in contiguous memory locations.
(b) A pointer when incremented always points to an immediately next location of its type.

Suppose we have an array **num[ ]** = { 24, 34, 12, 44, 56, 17 }. The following figure shows how this array is located in memory.

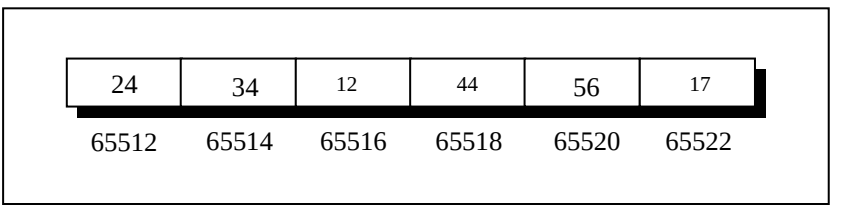


Figure 8.2

Here is a program that prints out the memory locations in which the elements of this array are stored.

```
main( )
{
    int  num[ ] = { 24, 34, 12, 44, 56, 17 } ;
    int  i ;

    for ( i = 0 ; i <= 5 ; i++ )
    {
        printf ( "\nelement no. %d ", i ) ;
        printf ( "address = %u", &num[i] ) ;
    }
}
```

The output of this program would look like this:

```
element no. 0 address = 65512
element no. 1 address = 65514
element no. 2 address = 65516
element no. 3 address = 65518
element no. 4 address = 65520
element no. 5 address = 65522
```

Note that the array elements are stored in contiguous memory locations, each element occupying two bytes, since it is an integer

array. When you run this program, you may get different addresses, but what is certain is that each subsequent address would be 2 bytes (4 bytes under Windows/Linux) greater than its immediate predecessor.

Our next two programs show ways in which we can access the elements of this array.

```
main( )
{
    int  num[ ] = { 24, 34, 12, 44, 56, 17 } ;
    int  i ;

    for ( i = 0 ; i <= 5 ; i++ )
    {
        printf ( "\naddress = %u ", &num[i] ) ;
        printf ( "element = %d", num[i] ) ;
    }
}
```

The output of this program would be:

```
address = 65512 element = 24
address = 65514 element = 34
address = 65516 element = 12
address = 65518 element = 44
address = 65520 element = 56
address = 65522 element = 17
```

This method of accessing array elements by using subscripted variables is already known to us. This method has in fact been given here for easy comparison with the next method, which accesses the array elements using pointers.

```
main( )
{
    int  num[ ] = { 24, 34, 12, 44, 56, 17 } ;
```

```
    int  i, *j ;

    j = &num[0] ;  /* assign address of zeroth element */

    for ( i = 0 ; i <= 5 ; i++ )
    {
        printf ( "\naddress = %u ", j ) ;
        printf ( "element = %d", *j ) ;
        j++ ;  /* increment pointer to point to next location */
    }
}
```

The output of this program would be:

```
address = 65512 element = 24
address = 65514 element = 34
address = 65516 element = 12
address = 65518 element = 44
address = 65520 element = 56
address = 65522 element = 17
```

In this program, to begin with we have collected the base address of the array (address of the 0^{th} element) in the variable **j** using the statement,

```
j = &num[0] ;  /* assigns address 65512 to j */
```

When we are inside the loop for the first time, **j** contains the address 65512, and the value at this address is 24. These are printed using the statements,

```
printf ( "\naddress = %u ", j ) ;
printf ( "element = %d", *j ) ;
```

On incrementing **j** it points to the next memory location of its type (that is location no. 65514). But location no. 65514 contains the second element of the array, therefore when the **printf( )**

statements are executed for the second time they print out the second element of the array and its address (i.e. 34 and 65514)... and so on till the last element of the array has been printed.

Obviously, a question arises as to which of the above two methods should be used when? Accessing array elements by pointers is **always** faster than accessing them by subscripts. However, from the point of view of convenience in programming we should observe the following:

Array elements should be accessed using pointers if the elements are to be accessed in a fixed order, say from beginning to end, or from end to beginning, or every alternate element or any such definite logic.

Instead, it would be easier to access the elements using a subscript if there is no fixed logic in accessing the elements. However, in this case also, accessing the elements by pointers would work faster than subscripts.

Passing an Entire Array to a Function

In the previous section we saw two programs—one in which we passed individual elements of an array to a function, and another in which we passed addresses of individual elements to a function. Let us now see how to pass an entire array to a function rather than its individual elements. Consider the following example:

```
/* Demonstration of passing an entire array to a function */
main( )
{
    int  num[ ] = { 24, 34, 12, 44, 56, 17 } ;
    dislpay ( &num[0], 6 ) ;
}

display ( int  *j, int  n )
{
```

```
    int  i ;

    for ( i = 0 ; i <= n - 1 ; i++ )
    {
        printf ( "\nelement = %d", *j ) ;
        j++ ;  /* increment pointer to point to next element */
    }
}
```

Here, the **display( )** function is used to print out the array elements. Note that the address of the zeroth element is being passed to the **display( )** function. The **for** loop is same as the one used in the earlier program to access the array elements using pointers. Thus, just passing the address of the zeroth element of the array to a function is as good as passing the entire array to the function. It is also necessary to pass the total number of elements in the array, otherwise the **display( )** function would not know when to terminate the **for** loop. Note that the address of the zeroth element (many a times called the base address) can also be passed by just passing the name of the array. Thus, the following two function calls are same:

```
display ( &num[0], 6 ) ;
display ( num, 6 ) ;
```

The Real Thing

If you have grasped the concept of storage of array elements in memory and the arithmetic of pointers, here is some real food for thought. Once again consider the following array.

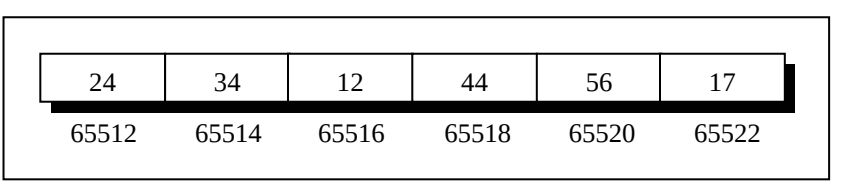


Figure 8.3

This is how we would declare the above array in C,

```
int  num[ ] = { 24, 34, 12, 44, 56, 17 } ;
```

We also know that on mentioning the name of the array we get its base address. Thus, by saying ***num** we would be able to refer to the zeroth element of the array, that is, 24. One can easily see that ***num** and ***(num + 0)** both refer to 24.

Similarly, by saying ***(num + 1)** we can refer the first element of the array, that is, 34. In fact, this is what the C compiler does internally. When we say, **num[i]**, the C compiler internally converts it to ***(num + i)**. This means that all the following notations are same:

```
num[i]
*( num + i )
*( i + num )
i[num]
```

And here is a program to prove my point.

```
/* Accessing array elements in different ways */
main( )
{
      int  num[ ] = { 24, 34, 12, 44, 56, 17 } ;
      int  i ;

      for ( i = 0 ; i <= 5 ; i++ )
      {
            printf ( "\naddress = %u ", &num[i] ) ;
            printf ( "element = %d %d ", num[i], *( num + i ) ) ;
            printf ( "%d %d", *( i + num ), i[num] ) ;
      }
}
```

The output of this program would be:

```
address = 65512 element = 24 24 24 24
address = 65514 element = 34 34 34 34
address = 65516 element = 12 12 12 12
address = 65518 element = 44 44 44 44
address = 65520 element = 56 56 56 56
address = 65522 element = 17 17 17 17
```

Two Dimensional Arrays

So far we have explored arrays with only one dimension. It is also possible for arrays to have two or more dimensions. The two-dimensional array is also called a matrix.

Here is a sample program that stores roll number and marks obtained by a student side by side in a matrix.

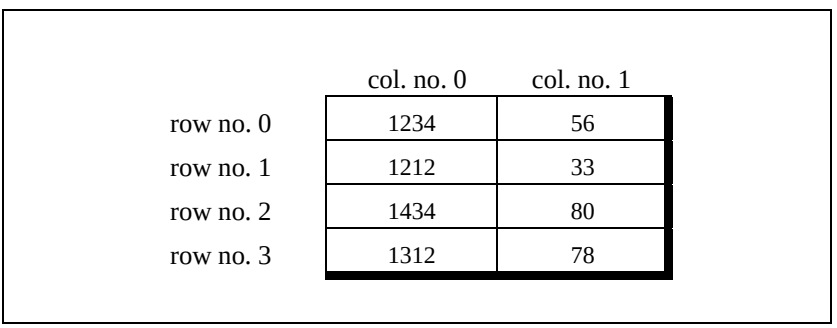
```
main( )
{
    int  stud[4][2] ;
    int  i, j ;

    for ( i = 0 ; i <= 3 ; i++ )
    {
        printf ( "\n Enter roll no. and marks" ) ;
        scanf ( "%d %d", &stud[i][0], &stud[i][1] ) ;
    }

    for ( i = 0 ; i <= 3 ; i++ )
        printf ( "\n%d %d", stud[i][0], stud[i][1] ) ;
}
```

There are two parts to the program—in the first part through a **for** loop we read in the values of roll no. and marks, whereas, in second part through another **for** loop we print out these values.

Look at the **scanf( )** statement used in the first **for** loop:

```
scanf ( "%d %d", &stud[i][0], &stud[i][1] ) ;
```

In **stud[i][0]** and **stud[i][1]** the first subscript of the variable **stud**, is row number which changes for every student. The second subscript tells which of the two columns are we talking about—the zeroth column which contains the roll no. or the first column which contains the marks. Remember the counting of rows and columns begin with zero. The complete array arrangement is shown below.



	col. no. 0	col. no. 1
row no. 0	1234	56
row no. 1	1212	33
row no. 2	1434	80
row no. 3	1312	78

Figure 8.4

Thus, 1234 is stored in **stud[0][0]**, 56 is stored in **stud[0][1]** and so on. The above arrangement highlights the fact that a two-dimensional array is nothing but a collection of a number of one-dimensional arrays placed one below the other.

In our sample program the array elements have been stored rowwise and accessed rowwise. However, you can access the array elements columnwise as well. Traditionally, the array elements are being stored and accessed rowwise; therefore we would also stick to the same strategy.

Initialising a 2-Dimensional Array

How do we initialize a two-dimensional array? As simple as this...

```
int  stud[4][2] = {
                    { 1234, 56 },
                    { 1212, 33 },
                    { 1434, 80 },
                    { 1312, 78 }
                  } ;
```

or even this would work...

```
int stud[4][2] = { 1234, 56, 1212, 33, 1434, 80, 1312, 78 } ;
```

of course with a corresponding loss in readability.

It is important to remember that while initializing a 2-D array it is necessary to mention the second (column) dimension, whereas the first dimension (row) is optional.

Thus the declarations,

```
int  arr[2][3] = { 12, 34, 23, 45, 56, 45 } ;
int  arr[ ][3] = { 12, 34, 23, 45, 56, 45 } ;
```

are perfectly acceptable,

whereas,

```
int  arr[2][ ] = { 12, 34, 23, 45, 56, 45 } ;
int  arr[ ][ ] = { 12, 34, 23, 45, 56, 45 } ;
```

would never work.

Memory Map of a 2-Dimensional Array

Let us reiterate the arrangement of array elements in a two-dimensional array of students, which contains roll nos. in one column and the marks in the other.

The array arrangement shown in Figure 8.4 is only conceptually true. This is because memory doesn't contain rows and columns. In memory whether it is a one-dimensional or a two-dimensional array the array elements are stored in one continuous chain. The arrangement of array elements of a two-dimensional array in memory is shown below:

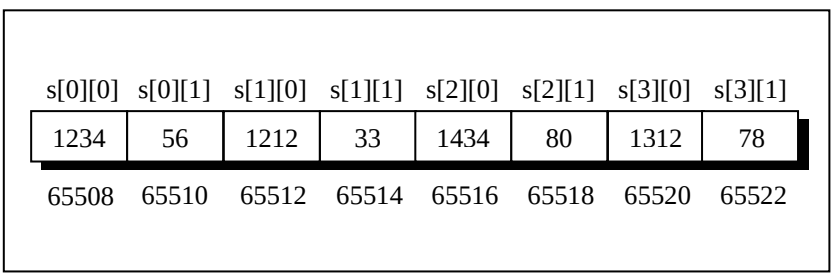


Figure 8.5

We can easily refer to the marks obtained by the third student using the subscript notation as shown below:

```
printf ( "Marks of third student = %d", stud[2][1] ) ;
```

Can we not refer the same element using pointer notation, the way we did in one-dimensional arrays? Answer is yes. Only the procedure is slightly difficult to understand. So, read on...

Pointers and 2-Dimensional Arrays

The C language embodies an unusual but powerful capability—it can treat parts of arrays as arrays. More specifically, each row of a two-dimensional array can be thought of as a one-dimensional array. This is a very important fact if we wish to access array elements of a two-dimensional array using pointers.

Thus, the declaration,

```
int  s[5][2] ;
```

can be thought of as setting up an array of 5 elements, each of which is a one-dimensional array containing 2 integers. We refer to an element of a one-dimensional array using a single subscript. Similarly, if we can imagine **s** to be a one-dimensional array then we can refer to its zeroth element as **s[0]**, the next element as **s[1]** and so on. More specifically, **s[0]** gives the address of the zeroth one-dimensional array, **s[1]** gives the address of the first one-dimensional array and so on. This fact can be demonstrated by the following program.

```
/* Demo: 2-D array is an array of arrays */
main( )
{
      int  s[4][2] = {
                              { 1234, 56 },
                              { 1212, 33 },
                              { 1434, 80 },
                              { 1312, 78 }
                        } ;
      int  i ;

      for ( i = 0 ; i <= 3 ; i++ )
            printf ( "\nAddress of %d th 1-D array = %u", i, s[i] ) ;
}
```

And here is the output...

```
Address of 0 th 1-D array = 65508
Address of 1 th 1-D array = 65512
Address of 2 th 1-D array = 65516
Address of 3 th 1-D array = 65520
```

Let's figure out how the program works. The compiler knows that **s** is an array containing 4 one-dimensional arrays, each containing 2 integers. Each one-dimensional array occupies 4 bytes (two bytes for each integer). These one-dimensional arrays are placed linearly (zeroth 1-D array followed by first 1-D array, etc.). Hence

each one-dimensional arrays starts 4 bytes further along than the last one, as can be seen in the memory map of the array shown below.

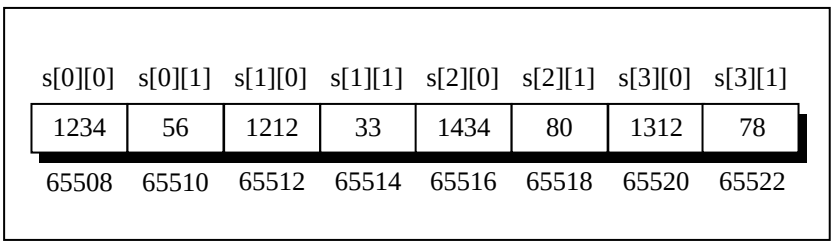


Figure 8.6

We know that the expressions **s[0]** and **s[1]** would yield the addresses of the zeroth and first one-dimensional array respectively. From Figure 8.6 these addresses turn out to be 65508 and 65512.

Now, we have been able to reach each one-dimensional array. What remains is to be able to refer to individual elements of a one-dimensional array. Suppose we want to refer to the element **s[2][1]** using pointers. We know (from the above program) that **s[2]** would give the address 65516, the address of the second one-dimensional array. Obviously (65516 + 1) would give the address 65518. Or **(s[2] + 1)** would give the address 65518. And the value at this address can be obtained by using the value at address operator, saying ***(s[2] + 1)**. But, we have already studied while learning one-dimensional arrays that **num[i]** is same as ***(num + i)**. Similarly, ***(s[2] + 1)** is same as, ***(*(s + 2) + 1)**. Thus, all the following expressions refer to the same element,

```
s[2][1]
*( s[2] + 1 )
*( *( s + 2 ) + 1 )
```

Using these concepts the following program prints out each element of a two-dimensional array using pointer notation.

```
/* Pointer notation to access 2-D array elements */
main( )
{
      int  s[4][2] = {
                              { 1234, 56 },
                              { 1212, 33 },
                              { 1434, 80 },
                              { 1312, 78 }
                        } ;
      int  i, j ;

      for ( i = 0 ; i <= 3 ; i++ )
      {
            printf ( "\n" ) ;
            for ( j = 0 ; j <= 1 ; j++ )
                  printf ( "%d ", *( *( s + i ) + j ) ) ;
      }
}
```

And here is the output...

```
1234  56
1212  33
1434  80
1312  78
```

Pointer to an Array

If we can have a pointer to an integer, a pointer to a float, a pointer to a char, then can we not have a pointer to an array? We certainly can. The following program shows how to build and use it.

```
/* Usage of pointer to an array */
main( )
{
      int  s[5][2] = {
                              { 1234, 56 },
                              { 1212, 33 },
                              { 1434, 80 },
                              { 1312, 78 }
                        } ;
      int  ( *p )[2] ;
      int  i, j, *pint ;

      for ( i = 0 ; i <= 3 ; i++ )
      {
            p = &s[i] ;
            pint = p ;
            printf ( "\n" ) ;
            for ( j = 0 ; j <= 1 ; j++ )
                  printf ( "%d ", *( pint + j ) ) ;
      }
}
```

And here is the output...

```
1234  56
1212  33
1434  80
1312  78
```

Here **p** is a pointer to an array of two integers. Note that the parentheses in the declaration of **p** are necessary. Absence of them would make **p** an array of 2 integer pointers. Array of pointers is covered in a later section in this chapter. In the outer **for** loop each time we store the address of a new one-dimensional array. Thus first time through this loop **p** would contain the address of the zeroth 1-D array. This address is then assigned to an integer pointer **pint**. Lastly, in the inner **for** loop using the pointer **pint** we

have printed the individual elements of the 1-D array to which **p** is pointing.

But why should we use a pointer to an array to print elements of a 2-D array. Is there any situation where we can appreciate its usage better? The entity pointer to an array is immensely useful when we need to pass a 2-D array to a function. This is discussed in the next section.

Passing 2-D Array to a Function

There are three ways in which we can pass a 2-D array to a function. These are illustrated in the following program.

```
/* Three ways of accessing a 2-D array */

main( )
{
    int  a[3][4] =  {
                        1, 2, 3, 4,
                        5, 6, 7, 8,
                        9, 0, 1, 6
                    } ;

    clrscr( ) ;
    display ( a, 3, 4 ) ;
    show ( a, 3, 4 ) ;
    print ( a, 3, 4 ) ;
}
display ( int  *q, int  row, int  col )
{
    int  i, j ;

    for ( i = 0 ; i < row ; i++ )
    {
        for ( j = 0 ; j < col ; j++ )
            printf ( "%d ", * ( q + i * col + j ) ) ;
```

```
            printf ( "\n" ) ;
        }
        printf ("\n" ) ;
}

show ( int  ( *q )[4], int  row, int  col )
{
        int  i, j ;
        int  *p ;

        for ( i = 0 ; i < row ; i++ )
        {
            p = q + i ;
            for ( j = 0 ; j < col ; j++ )
                printf ( "%d ", * ( p + j ) ) ;

            printf ( "\n" ) ;
        }
        printf ( "\n" ) ;
}

print ( int  q[ ][4], int  row, int  col )
{
        int  i, j ;

        for ( i = 0 ; i < row ; i++ )
        {
            for ( j = 0 ; j < col ; j++ )
                printf ( "%d ", q[i][j] ) ;
            printf ( "\n" ) ;
        }
        printf ( "\n" ) ;
}
```

And here is the output…

```
1 2 3 4
5 6 7 8
```

```
9 0 1 6

1 2 3 4
5 6 7 8
9 0 1 6

1 2 3 4
5 6 7 8
9 0 1 6
```

In the **display( )** function we have collected the base address of the 2-D array being passed to it in an ordinary **int** pointer. Then through the two **for** loops using the expression *** (q + i * col + j)** we have reached the appropriate element in the array. Suppose **i** is equal to 2 and **j** is equal to 3, then we wish to reach the element **a[2][3]**. Let us see whether the expression *** (q + i * col + j)** does give this element or not. Refer Figure 8.7 to understand this.

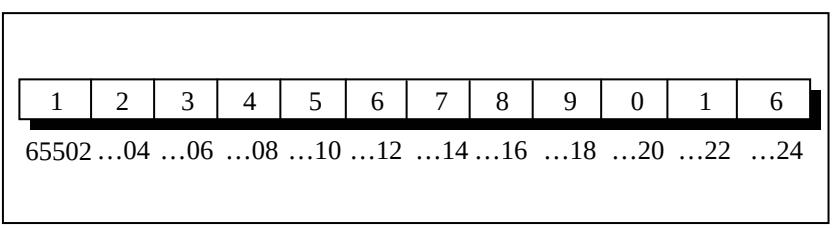


Figure 8.7

The expression *** (q + i * col + j)** becomes *** (65502 + 2 * 4 + 3)**. This turns out to be *** (65502 + 11).** Since **65502** is address of an integer, *** (65502 + 11)** turns out to be * (**65524)**. Value at this address is 6. This is indeed same as **a[2][3]**. A more general formula for accessing each array element would be:

* (base address + row no. * no. of columns + column no.)

In the **show( )** function we have defined **q** to be a pointer to an array of 4 integers through the declaration:

```
int  ( *q )[4] ;
```

To begin with, **q** holds the base address of the zeroth 1-D array, i.e. 4001 (refer Figure 8.7). This address is then assigned to **p**, an **int** pointer, and then using this pointer all elements of the zeroth 1-D array are accessed. Next time through the loop when **i** takes a value 1, the expression **q + i** fetches the address of the first 1-D array. This is because, **q** is a pointer to zeroth 1-D array and adding 1 to it would give us the address of the next 1-D array. This address is once again assigned to **p**, and using it all elements of the next 1-D array are accessed.

In the third function **print( )**, the declaration of **q** looks like this:

```
int  q[ ][4] ;
```

This is same as **int (*q)[4]**, where **q** is pointer to an array of 4 integers. The only advantage is that we can now use the more familiar expression **q[i][j]** to access array elements. We could have used the same expression in **show( )** as well.

Array of Pointers

The way there can be an array of **int**s or an array of **float**s, similarly there can be an array of pointers. Since a pointer variable always contains an address, an array of pointers would be nothing but a collection of addresses. The addresses present in the array of pointers can be addresses of isolated variables or addresses of array elements or any other addresses. All rules that apply to an ordinary array apply to the array of pointers as well. I think a program would clarify the concept.

```
main( )
{
    int  *arr[4] ;  /* array of integer pointers */
```

```
    int  i = 31, j = 5, k = 19, l = 71, m ;

    arr[0] = &i ;
    arr[1] = &j ;
    arr[2] = &k ;
    arr[3] = &l ;

    for ( m = 0 ; m  <= 3 ; m++ )
        printf ( "%d ", * ( arr[m] ) ) ;
}
```

Figure 8.8 shows the contents and the arrangement of the array of pointers in memory. As you can observe, **arr** contains addresses of isolated **int** variables **i**, **j**, **k** and **l**. The **for** loop in the program picks up the addresses present in **arr** and prints the values present at these addresses.

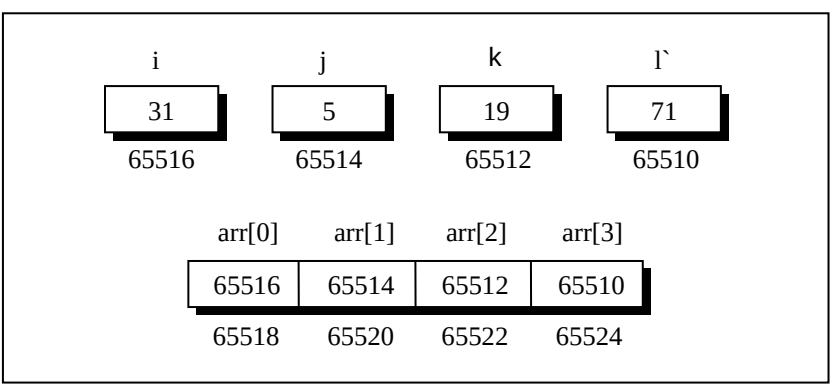


Figure 8.8

An array of pointers can even contain the addresses of other arrays. The following program would justify this.

```
main( )
{
    static int  a[ ] = { 0, 1, 2, 3, 4 } ;
```

```
    int  *p[ ] = { a, a + 1, a + 2, a + 3, a + 4 } ;

    printf ( "\n%u %u %d", p, *p, * ( *p ) ) ;
}
```

I would leave it for you to figure out the output of this program.

Three-Dimensional Array

We aren't going to show a programming example that uses a three-dimensional array. This is because, in practice, one rarely uses this array. However, an example of initializing a three-dimensional array will consolidate your understanding of subscripts:

```
int  arr[3][4][2] = {
                        {
                            { 2, 4 },
                            { 7, 8 },
                            { 3, 4 },
                            { 5, 6 }
                        },
                        {
                            { 7, 6 },
                            { 3, 4 },
                            { 5, 3 },
                            { 2, 3 }
                        },
                        {
                            { 8, 9 },
                            { 7, 2 },
                            { 3, 4 },
                            { 5, 1 },
                        }
                    } ;
```

A three-dimensional array can be thought of as an array of arrays of arrays. The outer array has three elements, each of which is a

two-dimensional array of four one-dimensional arrays, each of which contains two integers. In other words, a one-dimensional array of two elements is constructed first. Then four such one-dimensional arrays are placed one below the other to give a two-dimensional array containing four rows. Then, three such two-dimensional arrays are placed one behind the other to yield a three-dimensional array containing three 2-dimensional arrays. In the array declaration note how the commas have been given. Figure 8.9 would possibly help you in visualising the situation better.

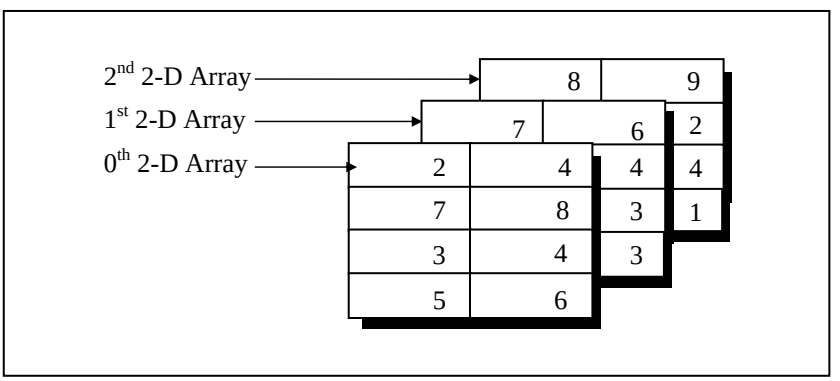


Figure 8.9

Again remember that the arrangement shown above is only conceptually true. In memory the same array elements are stored linearly as shown in Figure 8.10.

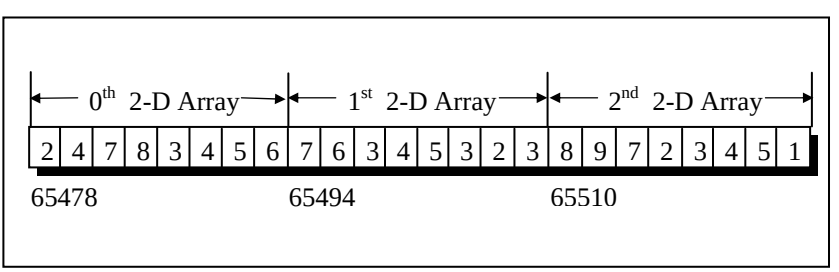


Figure 8.10

How would you refer to the array element 1 in the above array? The first subscript should be [2], since the element is in third two-dimensional array; the second subscript should be [3] since the element is in fourth row of the two-dimensional array; and the third subscript should be [1] since the element is in second position in the one-dimensional array. We can therefore say that the element 1 can be referred as **arr[2][3][1]**. It may be noted here that the counting of array elements even for a 3-D array begins with zero. Can we not refer to this element using pointer notation? Of course, yes. For example, the following two expressions refer to the same element in the 3-D array:

```
arr[2][3][1]
*( *( *( arr + 2 ) + 3 ) + 1 )
```

Summary

(a) An array is similar to an ordinary variable except that it can store multiple elements of similar type.
(b) Compiler doesn't perform bounds checking on an array.
(c) The array variable acts as a pointer to the zeroth element of the array. In a 1-D array, zeroth element is a single value, whereas, in a 2-D array this element is a 1-D array.
(d) On incrementing a pointer it points to the next location of its type.
(e) Array elements are stored in contiguous memory locations and so they can be accessed using pointers.
(f) Only limited arithmetic can be done on pointers.

Exercise

Simple arrays

[A] What would be the output of the following programs:

(a)
```
main( )
```

```
{
    int  num[26], temp ;
    num[0] = 100 ;
    num[25] = 200 ;
    temp = num[25] ;
    num[25] = num[0] ;
    num[0] = temp ;
    printf ( "\n%d %d", num[0], num[25] ) ;
}
```

(b)
```
main( )
{
    int  array[26], i ;
    for ( i = 0 ; i <= 25 ; i++ )
    {
        array[i] = 'A' + i ;
        printf ( "\n%d %c", array[i], array[i] ) ;
    }
}
```

(c)
```
main( )
{
    int  sub[50], i ;
    for ( i = 0 ; i <= 48 ; i++ ) ;
    {
        sub[i] = i ;
        printf ( "\n%d", sub[i] ) ;
    }
}
```

[B] Point out the errors, if any, in the following program segments:

(a)
```
/* mixed has some char and some int values */
int char  mixed[100] ;

main( )
{
    int  a[10], i ;
```

```
        for ( i = 1 ; i <= 10 ; i++ )
        {
            scanf ( "%d", a[i] ) ;
            printf ( "%d", a[i] ) ;
        }
    }
```

(b)
```
    main( )
    {
        int  size ;
        scanf ( "%d", &size ) ;
        int  arr[size] ;
        for ( i = 1 ; i <= size ; i++ )
        {
            scanf ( "%d", arr[i] ) ;
            printf ( "%d", arr[i] ) ;
        }
    }
```

(c)
```
    main( )
    {
        int  i, a = 2, b = 3 ;
        int  arr[ 2 + 3 ] ;
        for ( i = 0 ; i < a+b ; i++ )
        {
            scanf ( "%d", &arr[i] ) ;
            printf ( "\n%d", arr[i] ) ;
        }
    }
```

[C] Answer the following:

(a) An array is a collection of

1. different data types scattered throughout memory
2. the same data type scattered throughout memory
3. the same data type placed next to each other in memory
4. different data types placed next to each other in memory

(b) Are the following array declarations correct?

```
int  a (25) ;
int size = 10, b[size] ;
int c = {0,1,2} ;
```

(c) Which element of the array does this expression reference?

```
num[4]
```

(d) What is the difference between the 5's in these two expressions? (Select the correct answer)

```
int  num[5] ;
num[5] = 11 ;
```

1. first is particular element, second is type
2. first is array size, second is particular element
3. first is particular element, second is array size
4. both specify array size

(e) State whether the following statements are True or False:

1. The array **int num[26]** has twenty-six elements.
2. The expression **num[1]** designates the first element in the array
3. It is necessary to initialize the array at the time of declaration.
4. The expression **num[27]** designates the twenty-eighth element in the array.

[D] Attempt the following:

(a) Twenty-five numbers are entered from the keyboard into an array. The number to be searched is entered through the keyboard by the user. Write a program to find if the number to be searched is present in the array and if it is present, display the number of times it appears in the array.

(b) Twenty-five numbers are entered from the keyboard into an array. Write a program to find out how many of them are positive, how many are negative, how many are even and how many odd.

(c) Implement the Selection Sort, Bubble Sort and Insertion sort algorithms on a set of 25 numbers. (Refer Figure 8.11 for the logic of the algorithms)

- Selection sort
- Bubble Sort
- Insertion Sort

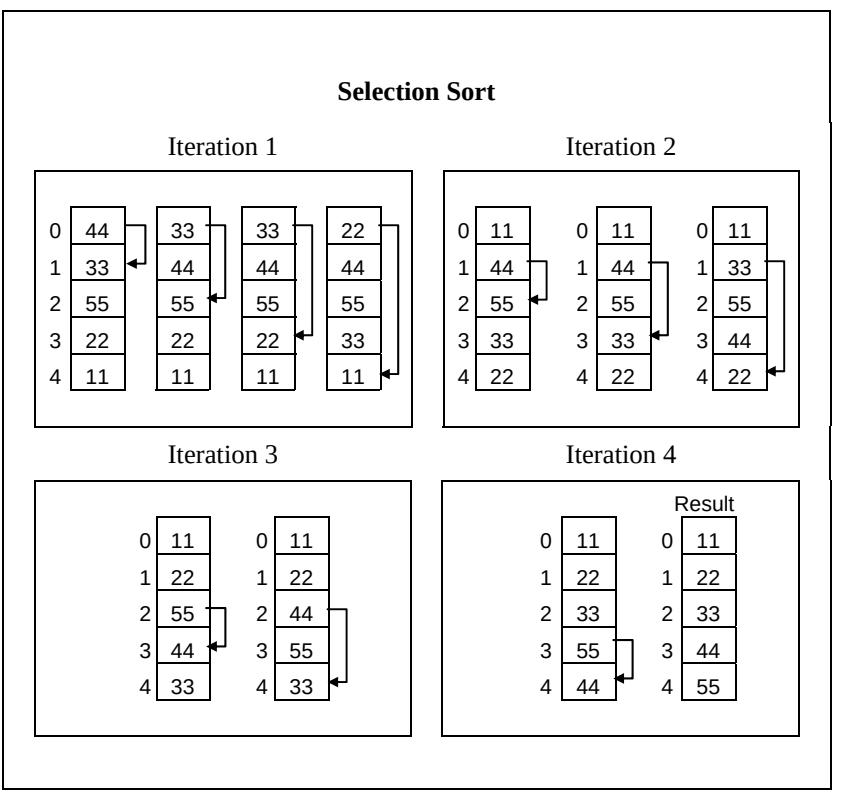


Figure 8.11 (a)

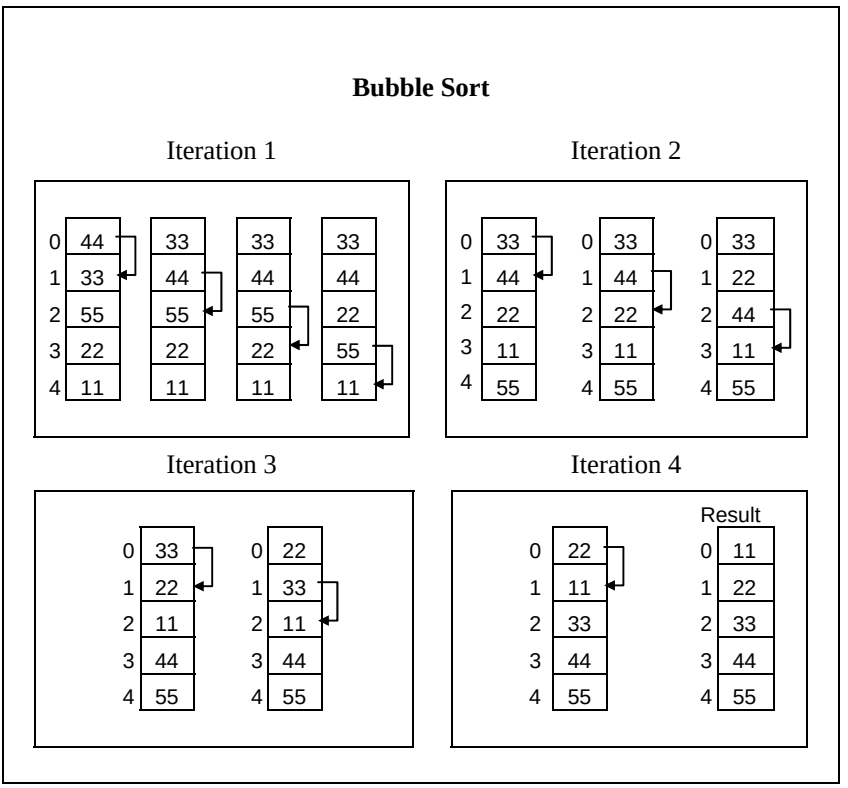


Figure 8.11 (b)

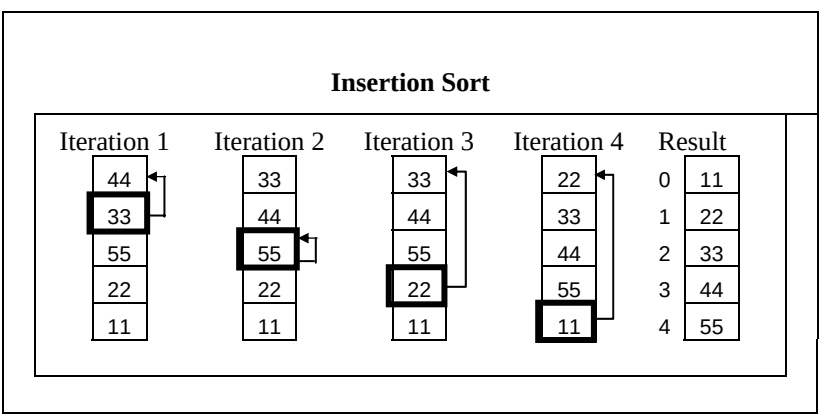


Figure 8.11 (c)

(d) Implement the following procedure to generate prime numbers from 1 to 100 into a program. This procedure is called sieve of Eratosthenes.

step 1 Fill an array **num[100]** with numbers from 1 to 100

step 2 Starting with the second entry in the array, set all its multiples to zero.

step 3 Proceed to the next non-zero element and set all its multiples to zero.

step 4 Repeat step 3 till you have set up the multiples of all the non-zero elements to zero

step 5 At the conclusion of step 4, all the non-zero entries left in the array would be prime numbers, so print out these numbers.

More on arrays, Arrays and pointers

[E] What would be the output of the following programs:

(a)
```
main( )
{
    int  b[ ] = { 10, 20, 30, 40, 50 } ;
    int  i ;
    for ( i = 0 ; i <= 4 ; i++ )
        printf ( "\n%d" *( b + i ) ) ;
}
```

(b)
```
main( )
{
    int  b[ ] = { 0, 20, 0, 40, 5 } ;
    int  i, *k ;
    k = b ;
    for ( i = 0 ; i <= 4 ; i++ )
    {
        printf ( "\n%d" *k ) ;
```

```
            k++ ;
        }
    }
```

(c)
```
main( )
{
    int  a[ ] = { 2, 4, 6, 8, 10 } ;
    int  i ;
    change ( a, 5 ) ;
    for ( i = 0 ; i <= 4 ; i++ )
        printf( "\n%d", a[i] ) ;
}
change ( int  *b, int  n )
{
    int  i ;
    for ( i = 0 ; i < n ; i++ )
        *( b + i ) = *( b + i ) + 5 ;
}
```

(d)
```
main( )
{
    int  a[5], i, b = 16 ;
    for ( i = 0 ; i < 5 ; i++ )
        a[i] = 2 * i ;
    f ( a, b ) ;
    for ( i = 0 ; i < 5 ; i++ )
        printf ( "\n%d", a[i] ) ;
    printf( "\n%d", b ) ;
}
f ( int  *x, int  y )
{
    int  i ;
    for ( i = 0 ; i < 5 ; i++ )
        *( x + i ) += 2 ;
    y += 2 ;
}
```

(e)
```
main( )
{
    static int  a[5] ;
    int  i ;
    for ( i = 0 ; i <= 4 ; i++ )
        printf ( "\n%d", a[i] ) ;
}
```

(f)
```
main( )
{
    int  a[5] = { 5, 1, 15, 20, 25 } ;
    int  i, j, k = 1, m ;
    i = ++a[1] ;
    j = a[1]++ ;
    m = a[i++] ;
    printf ( "\n%d %d %d", i, j, m ) ;
}
```

[F] Point out the errors, if any, in the following programs:

(a)
```
main( )
{
    int  array[6] = { 1, 2, 3, 4, 5, 6 } ;
    int  i ;
    for ( i = 0 ; i <= 25 ; i++ )
        printf ( "\n%d", array[i] ) ;
}
```

(b)
```
main( )
{
    int  sub[50], i ;
    for ( i = 1 ; i <= 50 ; i++ )
    {
        sub[i] = i ;
        printf ( "\n%d" , sub[i] ) ;
    }
}
```

(c)
```
main( )
{
    int  a[ ] = { 10, 20, 30, 40, 50 } ;
    int  j ;
    j = a ;  /* store the address of zeroth element */
    j = j + 3 ;
    printf ( "\n%d" *j ) ;
}
```

(d)
```
main( )
{
    float  a[ ] = { 13.24, 1.5, 1.5, 5.4, 3.5 } ;
    float  *j ;
    j = a ;
    j = j + 4 ;
    printf ( "\n%d %d %d", j, *j, a[4] ) ;
}
```

(e)
```
main( )
{
    float  a[ ] = { 13.24, 1.5, 1.5, 5.4, 3.5 } ;
    float  *j, *k ;
    j = a ;
    k = a + 4 ;
    j = j * 2 ;
    k = k / 2 ;
    printf ( "\n%d %d", *j, *k ) ;
}
```

(f)
```
main( )
{
    int  max = 5 ;
    float  arr[max] ;
    for ( i = 0 ; i < max ; i++ )
        scanf ( "%f", &arr[i] ) ;
}
```

[G] Answer the following:

(a) What would happen if you try to put so many values into an array when you initialize it that the size of the array is exceeded?

1. nothing
2. possible system malfunction
3. error message from the compiler
4. other data may be overwritten

(b) In an array **int arr[12]** the word **arr** represents the a_____ of the array

(c) What would happen if you put too few elements in an array when you initialize it?

1. nothing
2. possible system malfunction
3. error message from the compiler
4. unused elements will be filled with 0's or garbage

(d) What would happen if you assign a value to an element of an array whose subscript exceeds the size of the array?

1. the element will be set to 0
2. nothing, it's done all the time
3. other data may be overwritten
4. error message from the compiler

(e) When you pass an array as an argument to a function, what actually gets passed?

1. address of the array
2. values of the elements of the array
3. address of the first element of the array
4. number of elements of the array

(f) Which of these are reasons for using pointers?

1. To manipulate parts of an array
2. To refer to keywords such as **for** and **if**
3. To return more than one value from a function
4. To refer to particular programs more conveniently

(g) If you don't initialize a static array, what would be the elements set to?

1. 0
2. an undetermined value
3. a floating point number
4. the character constant '\0'

[H] State True or False:

(a) Address of a floating-point variable is always a whole number.

(b) Which of the following is the correct way of declaring a float pointer:

5. float ptr ;
6. float *ptr ;
7. *float ptr ;
8. None of the above

(c) Add the missing statement for the following program to print 35.

```
main( )
{
    int  j, *ptr ;
    *ptr = 35 ;
    printf ( "\n%d", j ) ;
}
```

(d) if **int s[5]** is a one-dimensional array of integers, which of the following refers to the third element in the array?

9. *(s + 2)
10. *(s + 3)
11. s + 3
12. s + 2

[I] Attempt the following:

(a) Write a program to copy the contents of one array into another in the reverse order.

(b) If an array **arr** contains **n** elements, then write a program to check if **arr[0] = arr[n-1]**, **arr[1] = arr[n-2]** and so on.

(c) Find the smallest number in an array using pointers.

(d) Write a program which performs the following tasks:

- initialize an integer array of 10 elements in **main( )**
- pass the entire array to a function **modify( )**
- in **modify( )** multiply each element of array by 3
- return the control to **main( )** and print the new array elements in **main( )**

(e) The screen is divided into 25 rows and 80 columns. The characters that are displayed on the screen are stored in a special memory called VDU memory (not to be confused with ordinary memory). Each character displayed on the screen occupies two bytes in VDU memory. The first of these bytes contains the ASCII value of the character being displayed, whereas, the second byte contains the colour in which the character is displayed.

For example, the ASCII value of the character present on zeroth row and zeroth column on the screen is stored at

location number 0xB8000000. Therefore the colour of this character would be present at location number 0xB8000000 + 1. Similarly ASCII value of character in row 0, col 1 will be at location 0xB8000000 + 2, and its colour at 0xB8000000 + 3.

With this knowledge write a program which when executed would keep converting every capital letter on the screen to small case letter and every small case letter to capital letter. The procedure should stop the moment the user hits a key from the keyboard.

This is an activity of a rampant Virus called Dancing Dolls. (For monochrome adapter, use 0xB0000000 instead of 0xB8000000).

More than one dimension

[J] What would be the output of the following programs:

(a)
```
main( )
{
    int  n[3][3] = {
                        2, 4, 3,
                        6, 8, 5,
                        3, 5, 1
                    } ;
    printf ( "\n%d %d %d", *n, n[3][3], n[2][2] ) ;
}
```

(b)
```
main( )
{
    int  n[3][3] = {
                        2, 4, 3,
                        6, 8, 5,
                        3, 5, 1
                    } ;
    int  i, *ptr ;
```

```
        ptr = n ;
        for ( i = 0 ; i <= 8 ; i++ )
            printf ( "\n%d", *( ptr + i ) ) ;
    }
```

(c)
```
    main( )
    {
        int  n[3][3] = {
                            2, 4, 3,
                            6, 8, 5,
                            3, 5, 1
                        } ;
        int  i, j ;
        for ( i = 0 ; i <= 2 ; i++ )
            for ( j = 0 ; j <= 2 ; j++ )
                printf ( "\n%d %d", n[i][j], *( *( n + i ) + j ) ) ;
    }
```

[K] Point out the errors, if any, in the following programs:

(a)
```
    main( )
    {
        int  twod[ ][ ] = {
                            2, 4,
                            6, 8
                          } ;
        printf ( "\n%d", twod ) ;
    }
```

(b)
```
    main( )
    {
        int  three[3][ ] = {
                            2, 4, 3,
                            6, 8, 2,
                            2, 3 ,1
                           } ;
        printf ( "\n%d", three[1][1] ) ;
```

}

[L] Attempt the following:

(a) How will you initialize a three-dimensional array **threed[3][2][3]**? How will you refer the first and last element in this array?

(b) Write a program to pick up the largest number from any 5 row by 5 column matrix.

(c) Write a program to obtain transpose of a 4 x 4 matrix. The transpose of a matrix is obtained by exchanging the elements of each row with the elements of the corresponding column.

(d) Very often in fairs we come across a puzzle that contains 15 numbered square pieces mounted on a frame. These pieces can be moved horizontally or vertically. A possible arrangement of these pieces is shown below:

1	4	15	7
8	10	2	11
14	3	6	13
12	9	5	

Figure 8.12

As you can see there is a blank at bottom right corner. Implement the following procedure through a program:

Draw the boxes as shown above. Display the numbers in the above order. Allow the user to hit any of the arrow keys (up, down, left, or right).

If user hits say, right arrow key then the piece with a number 5 should move to the right and blank should replace the original position of 5. Similarly, if down arrow key is hit, then 13 should move down and blank should replace the original position of 13. If left arrow key or up arrow key is hit then no action should be taken.

The user would continue hitting the arrow keys till the numbers aren't arranged in ascending order.

Keep track of the number of moves in which the user manages to arrange the numbers in ascending order. The user who manages it in minimum number of moves is the one who wins.

How do we tackle the arrow keys? We cannot receive them using **scanf( )** function. Arrow keys are special keys which are identified by their 'scan codes'. Use the following function in your program. It would return the scan code of the arrow key being hit. Don't worry about how this function is written. We are going to deal with it later. The scan codes for the arrow keys are:

up arrow key – 72 down arrow key – 80
left arrow key – 75 right arrow key – 77

```
/* Returns scan code of the key that has been hit */
#include "dos.h"
getkey( )
{
    union REGS i, o ;
```

```
    while ( !kbhit( ) )
          ;
    i.h.ah = 0 ;
    int86 ( 22, &i, &o ) ;
    return ( o.h.ah ) ;
}
```

(e) Those readers who are from an Engineering/Science background may try writing programs for following problems.

(1) Write a program to add two 6 x 6 matrices.
(2) Write a program to multiply any two 3 x 3 matrices.
(3) Write a program to sort all the elements of a 4 x 4 matrix.
(4) Write a program to obtain the determinant value of a 5 x 5 matrix.

(f) Match the following with reference to the following program segment:

```
int  i, j, = 25;
int  *pi, *pj = & j;
…….
…….  /* more lines of program */
…….
*pj = j + 5;
j = *pj + 5 ;
pj = pj ;
*pi = i + j
```

Each integer quantity occupies 2 bytes of memory. The value assigned to **i** begin at (hexadecimal) address F9C and the value assigned to j begins at address F9E. Match the value represented by left hand side quantities with the right.

1.	&i	a.	30
2.	&j	b.	F9E
3.	pj	c.	35
4.	*pj	d.	FA2

5.	i	e.	F9C
6.	pi	f.	67
7.	*pi	g.	unspecified
8.	(pi + 2)	h.	65
9.	(*pi + 2)	i.	F9E
10.	* (pi + 2)	j.	F9E
		k.	FAO
		l.	F9D

(g) Match the following with reference to the following segment:

```
int  x[3][5] = {
                    { 1, 2, 3, 4, 5 },
                    { 6, 7, 8, 9, 10 },
                    { 11, 12, 13, 14, 15 }
               }, *n = &x ;
```

1.	*(*(x + 2) + 1)	a.	9
2.	*(*x + 2) + 5	b.	13
3.	*(*(x + 1))	c.	4
4.	*(*(x) + 2) + 1	d.	3
5.	* (*(x + 1) + 3)	e.	2
6.	*n	f.	12
7.	*(n +2)	g.	14
8.	(*(n + 3) + 1	h.	7
9.	*(n + 5)+1	i.	1
10.	++*n	j.	8
		k.	5
		l.	10
		m.	6

(h) Match the following with reference to the following program segment:

```
struct
{
    int  x, y;
} s[ ] = { 10, 20, 15, 25, 8, 75, 6, 2 };
int  *i ;
i = s ;
```

1.	*(i + 3)	a.	85
2.	s[i[7]].x	b.	2
3.	s[(s + 2)->y / 3[I]].y	c.	6
4.	i[i[1]-i[2]]	d.	7
5.	i[s[3].y]	e.	16
6.	(s + 1)->x + 5	f.	15
7.	*(1 +i)**(i + 4) / *i	g.	25
8.	s[i[0] – i[4]].y + 10	h.	8
9.	(*(s + *(i + 1) / *i)).x + 2	i.	1
10.	++i[i[6]]	j.	100
		k.	10
		l.	20

(i) Match the following with reference to the following program segment:

```
unsigned int  arr[3][3] = {
                              2, 4, 6,
                              9, 1, 10,
                              16, 64, 5
                          } ;
```

1.	**arr	a.	64
2.	**arr < *(*arr + 2)	b.	18
3.	*(arr + 2) / (*(*arr + 1) > **arr)	c.	6
4.	*(arr[1] + 1) | arr[1][2]	d.	3
5.	*(arr[0]) | *(arr[2])	e.	0
6.	arr[1][1] < arr[0][1]	f.	16
7.	arr[2][[1] & arr[2][0]	g.	1
8.	arr[2][2] | arr[0][1]	h.	11
9.	arr[0][1] ^ arr[0][2]	i.	20
10.	++**arr + --arr[1][1]	j.	2
		k.	5
		l.	4

(j) Write a program that interchanges the odd and even components of an array.

(k) Write a program to find if a square matrix is symmetric.

(l) Write a function to find the norm of a matrix. The norm is defined as the square root of the sum of squares of all elements in the matrix.

(m) Given an array **p[5]**, write a function to shift it circularly left by two positions. Thus, if p[0] = 15, p[1]= 30, p[2] = 28, p[3]= 19 and p[4] = 61 then after the shift p[0] = 28, p[1] = 19, p[2] = 61, p[3] = 15 and p[4] = 30. Call this function for a (4 x 5) matrix and get its rows left shifted.

(n) A 6 x 6 matrix is entered through the keyboard and stored in a 2-dimensional array **mat[7][7]**. Write a program to obtain the Determinant values of this matrix.

(o) For the following set of sample data, compute the standard deviation and the mean.

-6, -12, 8, 13, 11, 6, 7, 2, -6, -9, -10, 11, 10, 9, 2

The formula for standard deviation is

$$\frac{\sqrt{(x_i - \bar{x})^2}}{n}$$

where $\mathbf{x_i}$ is the data item and $\bar{\mathbf{x}}$ is the mean.

(p) The area of a triangle can be computed by the sine law when 2 sides of the triangle and the angle between them are known.

Area = (1 / 2) ab sin (angle)

Given the following 6 triangular pieces of land, write a program to find their area and determine which is largest,

Plot No.	a	b	angle
1	137.4	80.9	0.78
2	155.2	92.62	0.89
3	149.3	97.93	1.35

4	160.0	100.25	9.00
5	155.6	68.95	1.25
6	149.7	120.0	1.75

(q) For the following set of n data points (x, y), compute the correlation coefficient r, given by

$$r = \frac{\sum xy - \sum x \sum y}{\sqrt{[n\sum x^2 - (\sum x)^2][n\sum y^2 - (\sum y)^2]}}$$

x	y
34.22	102.43
39.87	100.93
41.85	97.43
43.23	97.81
40.06	98.32
53.29	98.32
53.29	100.07
54.14	97.08
49.12	91.59
40.71	94.85
55.15	94.65

(r) For the following set of point given by **(x, y)** fit a straight line given by

y = a + bx

where,

$a = \bar{y} - b\bar{x}$ and

$$b = \frac{n\sum yx - \sum x \sum y}{[n\sum x^2 - (\sum x)^2]}$$

x	y
3.0	1.5

4.5	2.0
5.5	3.5
6.5	5.0
7.5	6.0
8.5	7.5
8.0	9.0
9.0	10.5
9.5	12.0
10.0	14.0

(s) The **X** and **Y** coordinates of 10 different points are entered through the keyboard. Write a program to find the distance of last point from the first point (sum of distance between consecutive points).

9 Puppetting On Strings

- What are Strings
- More about Strings
- Pointers and Strings
- Standard Library String Functions
  - *strlen( )*
  - *strcpy*( )
  - *strcat*( )
  - *strcmp*( )
- Two-Dimensional Array of Characters
- Array of Pointers to Strings
- Limitation of Array of Pointers to Strings
  - Solution
- Summary
- Exercise

In the last chapter you learnt how to define arrays of differing sizes and dimensions, how to initialize arrays, how to pass arrays to a function, etc. With this knowledge under your belt, you should be ready to handle strings, which are, simply put, a special kind of array. And strings, the ways to manipulate them, and how pointers are related to strings are going to be the topics of discussion in this chapter.

What are Strings

The way a group of integers can be stored in an integer array, similarly a group of characters can be stored in a character array. Character arrays are many a time also called strings. Many languages internally treat strings as character arrays, but somehow conceal this fact from the programmer. Character arrays or strings are used by programming languages to manipulate text such as words and sentences.

A string constant is a one-dimensional array of characters terminated by a null ('\0'). For example,

```
char  name[ ] = { 'H', 'A', 'E', 'S', 'L', 'E', 'R', '\0' } ;
```

Each character in the array occupies one byte of memory and the last character is always '\0'. What character is this? It looks like two characters, but it is actually only one character, with the \ indicating that what follows it is something special. '\0' is called null character. Note that '\0' and '0' are not same. ASCII value of '\0' is 0, whereas ASCII value of '0' is 48. Figure 9.1 shows the way a character array is stored in memory. Note that the elements of the character array are stored in contiguous memory locations.

The terminating null ('\0') is important, because it is the only way the functions that work with a string can know where the string ends. In fact, a string not terminated by a '\0' is not really a string, but merely a collection of characters.

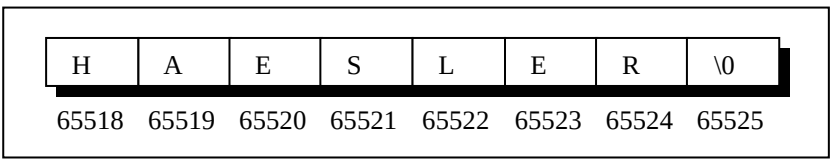


Figure 9.1

C concedes the fact that you would use strings very often and hence provides a shortcut for initializing strings. For example, the string used above can also be initialized as,

```
char  name[ ] = "HAESLER" ;
```

Note that, in this declaration '\0' is not necessary. C inserts the null character automatically.

More about Strings

In what way are character arrays different than numeric arrays? Can elements in a character array be accessed in the same way as the elements of a numeric array? Do I need to take any special care of '\0'? Why numeric arrays don't end with a '\0'? Declaring strings is okay, but how do I manipulate them? Questions galore!! Well, let us settle some of these issues right away with the help of sample programs.

```
/* Program to demonstrate printing of a string */
main( )
{
      char  name[ ] = "Klinsman" ;
      int  i = 0 ;

      while ( i <= 7 )
      {
            printf ( "%c", name[i] ) ;
            i++ ;
      }
```

```
}
```

And here is the output...

```
Klinsman
```

No big deal. We have initialized a character array, and then printed out the elements of this array within a **while** loop. Can we write the **while** loop without using the final value 7? We can; because we know that each character array always ends with a '\0'. Following program illustrates this.

```
main( )
{
    char  name[ ] = "Klinsman" ;
    int  i = 0 ;

    while ( name[i] != `\0' )
    {
        printf ( "%c", name[i] ) ;
        i++ ;
    }
}
```

And here is the output...

```
Klinsman
```

This program doesn't rely on the length of the string (number of characters in it) to print out its contents and hence is definitely more general than the earlier one. Here is another version of the same program; this one uses a pointer to access the array elements.

```
main( )
{
    char  name[ ] = "Klinsman" ;
    char  *ptr ;
```

```
    ptr = name ;  /* store base address of string */

    while ( *ptr != `\0' )
    {
        printf ( "%c", *ptr ) ;
        ptr++ ;
    }
}
```

As with the integer array, by mentioning the name of the array we get the base address (address of the zeroth element) of the array. This base address is stored in the variable **ptr** using,

```
ptr = name ;
```

Once the base address is obtained in **ptr**, ***ptr** would yield the value at this address, which gets printed promptly through,

```
printf ( "%c", *ptr ) ;
```

Then, **ptr** is incremented to point to the next character in the string. This derives from two facts: array elements are stored in contiguous memory locations and on incrementing a pointer it points to the immediately next location of its type. This process is carried out till **ptr** doesn't point to the last character in the string, that is, '\0'.

In fact, the character array elements can be accessed exactly in the same way as the elements of an integer array. Thus, all the following notations refer to the same element:

```
name[i]
*( name + i )
*( i + name )
i[name]
```

Even though there are so many ways (as shown above) to refer to the elements of a character array, rarely is any one of them used. This is because **printf( )** function has got a sweet and simple way of doing it, as shown below. Note that **printf( )** doesn't print the '\0'.

```
main( )
{
    char  name[ ] = "Klinsman" ;
    printf ( "%s", name ) ;
}
```

The **%s** used in **printf( )** is a format specification for printing out a string. The same specification can be used to receive a string from the keyboard, as shown below.

```
main( )
{
    char  name[25] ;

    printf ( "Enter your name " ) ;
    scanf ( "%s", name ) ;
    printf ( "Hello %s!", name ) ;
}
```

And here is a sample run of the program...

```
Enter your name Debashish
Hello Debashish!
```

Note that the declaration **char name[25]** sets aside 25 bytes under the array **name[ ],** whereas the **scanf( )** function fills in the characters typed at keyboard into this array until the enter key is hit. Once enter is hit, **scanf( )** places a '\0' in the array. Naturally, we should pass the base address of the array to the **scanf( )** function.

While entering the string using **scanf( )** we must be cautious about two things:

(a) The length of the string should not exceed the dimension of the character array. This is because the C compiler doesn't perform bounds checking on character arrays. Hence, if you carelessly exceed the bounds there is always a danger of overwriting something important, and in that event, you would have nobody to blame but yourselves.

(b) **scanf( )** is not capable of receiving multi-word strings. Therefore names such as 'Debashish Roy' would be unacceptable. The way to get around this limitation is by using the function **gets( )**. The usage of functions **gets( )** and its counterpart **puts( )** is shown below.

```
main( )
{
    char  name[25] ;

    printf ( "Enter your full name " ) ;
    gets ( name ) ;
    puts ( "Hello!" ) ;
    puts ( name ) ;
}
```

And here is the output...

```
Enter your name Debashish Roy
Hello!
Debashish Roy
```

The program and the output are self-explanatory except for the fact that, **puts( )** can display only one string at a time (hence the use of two **puts( )** in the program above). Also, on displaying a string, unlike **printf( )**, **puts( )** places the cursor on the next line. Though **gets( )** is capable of receiving only

one string at a time, the plus point with **gets( )** is that it can receive a multi-word string.

If we are prepared to take the trouble we can make **scanf( )** accept multi-word strings by writing it in this manner:

```
char  name[25] ;
printf ( "Enter your full name " ) ;
scanf ( "%[^\n]s", name ) ;
```

Though workable this is the best of the ways to call a function, you would agree.

Pointers and Strings

Suppose we wish to store “Hello”. We may either store it in a string or we may ask the C compiler to store it at some location in memory and assign the address of the string in a **char** pointer. This is shown below:

```
char  str[ ] = "Hello" ;
char  *p = "Hello" ;
```

There is a subtle difference in usage of these two forms. For example, we cannot assign a string to another, whereas, we can assign a **char** pointer to another **char** pointer. This is shown in the following program.

```
main( )
{
    char  str1[ ] = "Hello" ;
    char  str2[10] ;

    char  *s = "Good Morning" ;
    char  *q ;
```

```
    str2 = str1 ; /* error */
    q = s ; /* works */
}
```

Also, once a string has been defined it cannot be initialized to another set of characters. Unlike strings, such an operation is perfectly valid with **char** pointers.

```
main( )
{
    char  str1[ ] = "Hello" ;
    char  *p = "Hello" ;
    str1 = "Bye" ; /* error */
    p = "Bye" ; /* works */
}
```

Standard Library String Functions

With every C compiler a large set of useful string handling library functions are provided. Figure 9.2 lists the more commonly used functions along with their purpose.

Function	Use
strlen	Finds length of a string
strlwr	Converts a string to lowercase
strupr	Converts a string to uppercase
strcat	Appends one string at the end of another
strncat	Appends first n characters of a string at the end of another

strcpy	Copies a string into another
strncpy	Copies first n characters of one string into another
strcmp	Compares two strings
strncmp	Compares first n characters of two strings
strcmpi	Compares two strings without regard to case ("i" denotes that this function ignores case)
stricmp	Compares two strings without regard to case (identical to strcmpi)
strnicmp	Compares first n characters of two strings without regard to case
strdup	Duplicates a string
strchr	Finds first occurrence of a given character in a string
strrchr	Finds last occurrence of a given character in a string
strstr	Finds first occurrence of a given string in another string
strset	Sets all characters of string to a given character
strnset	Sets first n characters of a string to a given character
strrev	Reverses string

Figure 9.2

Out of the above list we shall discuss the functions **strlen( )**, **strcpy( )**, **strcat( )** and **strcmp( )**, since these are the most commonly used functions. This will also illustrate how the library functions in general handle strings. Let us study these functions one by one.

strlen( )

This function counts the number of characters present in a string. Its usage is illustrated in the following program.

```
main( )
{
    char  arr[ ] = "Bamboozled" ;
    int  len1, len2 ;
```

```
    len1 = strlen ( arr ) ;
    len2 = strlen ( "Humpty Dumpty" ) ;

    printf ( "\nstring = %s length = %d", arr, len1 ) ;
    printf ( "\nstring = %s length = %d", "Humpty Dumpty", len2 ) ;
}
```

The output would be...

```
string = Bamboozled length = 10
string = Humpty Dumpty length = 13
```

Note that in the first call to the function **strlen( )**, we are passing the base address of the string, and the function in turn returns the length of the string. While calculating the length it doesn't count '\0'. Even in the second call,

```
len2 = strlen ( "Humpty Dumpty" ) ;
```

what gets passed to **strlen( )** is the address of the string and not the string itself. Can we not write a function **xstrlen( )** which imitates the standard library function **strlen( )**? Let us give it a try...

```
/* A look-alike of the function strlen( ) */
main( )
{
    char  arr[ ] = "Bamboozled" ;
    int  len1, len2 ;

    len1 = xstrlen ( arr ) ;
    len2 = xstrlen ( "Humpty Dumpty" ) ;

    printf ( "\nstring = %s length = %d", arr, len1 ) ;
    printf ( "\nstring = %s length = %d", "Humpty Dumpty", len2 ) ;
}
```

```
xstrlen ( char  *s )
{
      int  length = 0 ;

      while ( *s != '\0' )
      {
            length++ ;
            s++ ;
      }

      return ( length ) ;
}
```

The output would be...

```
string = Bamboozled length = 10
string = Humpty Dumpty length = 13
```

The function **xstrlen( )** is fairly simple. All that it does is keep counting the characters till the end of string is not met. Or in other words keep counting characters till the pointer s doesn't point to '\0'.

strcpy( )

This function copies the contents of one string into another. The base addresses of the source and target strings should be supplied to this function. Here is an example of **strcpy( )** in action...

```
main( )
{
      char  source[ ] = "Sayonara" ;
```

```
    char  target[20] ;

    strcpy ( target, source ) ;
    printf ( "\nsource string = %s", source ) ;
    printf ( "\ntarget string = %s", target ) ;
}
```

And here is the output...

```
source string = Sayonara
target string = Sayonara
```

On supplying the base addresses, **strcpy( )** goes on copying the characters in source string into the target string till it doesn't encounter the end of source string ('\0'). It is our responsibility to see to it that the target string's dimension is big enough to hold the string being copied into it. Thus, a string gets copied into another, piece-meal, character by character. There is no short cut for this. Let us now attempt to mimic **strcpy( )**, via our own string copy function, which we will call **xstrcpy( ).**

```
main( )
{
    char  source[ ] = "Sayonara" ;
    char  target[20] ;

    xstrcpy ( target, source ) ;
    printf ( "\nsource string = %s", source ) ;
    printf ( "\ntarget string = %s", target ) ;
}

xstrcpy ( char  *t, char  *s )
{
    while ( *s != '\0' )
    {
        *t = *s ;
        s++ ;
```

```
        t++ ;
    }
    *t = '\0' ;
}
```

The output of the program would be...

```
source string = Sayonara
target string = Sayonara
```

Note that having copied the entire source string into the target string, it is necessary to place a '\0' into the target string, to mark its end.

If you look at the prototype of **strcpy( )** standard library function, it looks like this…

```
strcpy ( char *t, const char *s ) ;
```

We didn't use the keyword **const** in our version of **xstrcpy( )** and still our function worked correctly. So what is the need of the **const** qualifier?

What would happen if we add the following lines beyond the last statement of **xstrcpy( )**?.

```
s = s - 8 ;
*s = 'K' ;
```

This would change the source string to "Kayonara". Can we not ensure that the source string doesn't change even accidentally in **xstrcpy( )**? We can, by changing the definition as follows:

```
void xstrcpy ( char *t, const char *s )
{
    while ( *s != '\0' )
    {
```

```
            *t = *s ;
            s++ ;
            t++ ;
        }
        *t = '\0' ;
}
```

By declaring **char *s** as **const** we are declaring that the source string should remain constant (should not change). Thus the **const** qualifier ensures that your program does not inadvertently alter a variable that you intended to be a constant. It also reminds anybody reading the program listing that the variable is not intended to change.

We can use **const** in several situations. The following code fragment would help you to fix your ideas about **const** further.

```
char *p = "Hello" ;  /* pointer is variable, so is string */
*p = 'M' ;  /* works */
p = "Bye" ;  /* works */

const char *q = "Hello" ;  /* string is fixed pointer is not */
*q = 'M' ;  /* error */
q = "Bye" ;  /* works */

char const *s = "Hello" ;  /* string is fixed pointer is not */
*s = 'M' ;  /* error */
s = "Bye" ;  /* works */

char * const t = "Hello" ;  /* pointer is fixed string is not */
*t = 'M' ;  /* works */
t = "Bye" ;  /* error */

const char * const u = "Hello" ;  /* string is fixed so is pointer */
*u = 'M' ;  /* error */
u = "Bye" ;  /* error */
```

The keyword **const** can be used in context of ordinary variables like **int**, **float**, etc. The following program shows how this can be done.

```
main( )
{
      float r, a ;
      const float pi = 3.14 ;

      printf ( "\nEnter radius of circle " ) ;
      scanf  ( "%f", &r ) ;
      a = pi * r * r ;
      printf ( "\nArea of circle = %f", a ) ;
}
```

strcat( )

This function concatenates the source string at the end of the target string. For example, “Bombay” and “Nagpur” on concatenation would result into a string “BombayNagpur”. Here is an example of **strcat( )** at work.

```
main( )
{
      char  source[ ] = "Folks!" ;
      char  target[30] = "Hello" ;

      strcat ( target, source ) ;
      printf ( "\nsource string = %s", source ) ;
      printf ( "\ntarget string = %s", target ) ;
}
```

And here is the output...

```
source string = Folks!
target string = HelloFolks!
```

Note that the target string has been made big enough to hold the final string. I leave it to you to develop your own **xstrcat( )** on lines of **xstrlen( )** and **xstrcpy( ).**

strcmp( )

This is a function which compares two strings to find out whether they are same or different. The two strings are compared character by character until there is a mismatch or end of one of the strings is reached, whichever occurs first. If the two strings are identical, **strcmp( )** returns a value zero. If they're not, it returns the numeric difference between the ASCII values of the first non-matching pairs of characters. Here is a program which puts **strcmp( )** in action.

```
main( )
{
    char  string1[ ] = "Jerry" ;
    char  string2[ ] = "Ferry" ;
    int  i, j, k ;

    i = strcmp ( string1, "Jerry" ) ;
    j = strcmp ( string1, string2 ) ;
    k = strcmp ( string1, "Jerry boy" ) ;

    printf ( "\n%d %d %d", i, j, k ) ;
}
```

And here is the output...

```
0 4 -32
```

In the first call to **strcmp( )**, the two strings are identical—"Jerry" and "Jerry"—and the value returned by **strcmp( )** is zero. In the second call, the first character of "Jerry" doesn't match with the first character of "Ferry" and the result is 4, which is the numeric

difference between ASCII value of 'J' and ASCII value of 'F'. In the third call to **strcmp( )** "Jerry" doesn't match with "Jerry boy", because the null character at the end of "Jerry" doesn't match the blank in "Jerry boy". The value returned is -32, which is the value of null character minus the ASCII value of space, i.e., '\0' minus ' ', which is equal to -32.

The exact value of mismatch will rarely concern us. All we usually want to know is whether or not the first string is alphabetically before the second string. If it is, a negative value is returned; if it isn't, a positive value is returned. Any non-zero value means there is a mismatch. Try to implement this procedure into a function **xstrcmp( )**.

Two-Dimensional Array of Characters

In the last chapter we saw several examples of 2-dimensional integer arrays. Let's now look at a similar entity, but one dealing with characters. Our example program asks you to type your name. When you do so, it checks your name against a master list to see if you are worthy of entry to the palace. Here's the program...

```
#define FOUND 1
#define NOTFOUND 0
main( )
{
    char  masterlist[6][10] = {
                                "akshay",
                                "parag",
                                "raman",
                                "srinivas",
                                "gopal",
                                "rajesh"
                              } ;
    int  i, flag, a ;
    char  yourname[10] ;
```

```
    printf ( "\nEnter your name " ) ;
    scanf ( "%s", yourname ) ;

    flag = NOTFOUND ;
    for ( i = 0 ; i <= 5 ; i++ )
    {
        a = strcmp ( &masterlist[i][0], yourname ) ;
        if ( a == 0 )
        {
            printf ( "Welcome, you can enter the palace" ) ;
            flag = FOUND ;
            break ;
        }
    }

    if ( flag == NOTFOUND )
        printf ( "Sorry, you are a trespasser" ) ;
}
```

And here is the output for two sample runs of this program...

```
Enter your name dinesh
Sorry, you are a trespasser
Enter your name raman
Welcome, you can enter the palace
```

Notice how the two-dimensional character array has been initialized. The order of the subscripts in the array declaration is important. The first subscript gives the number of names in the array, while the second subscript gives the length of each item in the array.

Instead of initializing names, had these names been supplied from the keyboard, the program segment would have looked like this...

```
for ( i = 0 ; i <= 5 ; i++ )
    scanf ( "%s", &masterlist[i][0] ) ;
```

While comparing the strings through **strcmp( )**, note that the addresses of the strings are being passed to **strcmp( )**. As seen in the last section, if the two strings match, **strcmp( )** would return a value 0, otherwise it would return a non-zero value.

The variable **flag** is used to keep a record of whether the control did reach inside the **if** or not. To begin with, we set **flag** to NOTFOUND. Later through the loop if the names match, **flag** is set to FOUND. When the control reaches beyond the **for** loop, if **flag** is still set to NOTFOUND, it means none of the names in the **masterlist[ ][ ]** matched with the one supplied from the keyboard.

The names would be stored in the memory as shown in Figure 9.3. Note that each string ends with a '\0'. The arrangement as you can appreciate is similar to that of a two-dimensional numeric array.

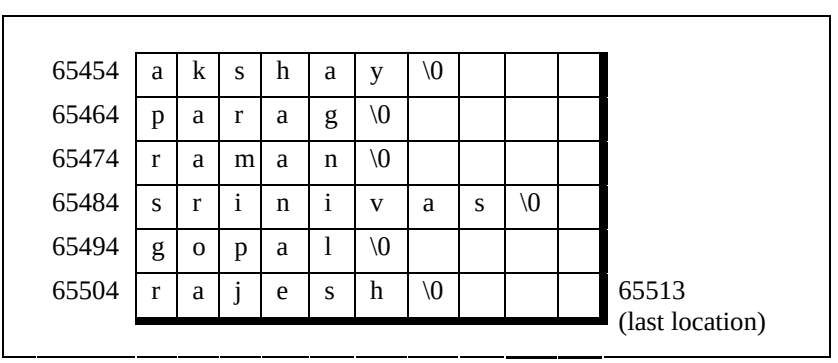


Figure 9.3

Here, 65454, 65464, 65474, etc. are the base addresses of successive names. As seen from the above pattern some of the names do not occupy all the bytes reserved for them. For example, even though 10 bytes are reserved for storing the name “akshay”, it occupies only 7 bytes. Thus, 3 bytes go waste. Similarly, for each name there is some amount of wastage. In fact, more the number of names, more would be the wastage. Can this not be avoided? Yes, it can be... by using what is called an ‘array of pointers’, which is our next topic of discussion.

Array of Pointers to Strings

As we know, a pointer variable always contains an address. Therefore, if we construct an array of pointers it would contain a number of addresses. Let us see how the names in the earlier example can be stored in the array of pointers.

```
char  *names[ ] = {
                        "akshay",
                        "parag",
                        "raman",
                        "srinivas",
                        "gopal",
                        "rajesh"
                  } ;
```

In this declaration **names[ ]** is an array of pointers. It contains base addresses of respective names. That is, base address of “akshay” is stored in **names[0]**, base address of “parag” is stored in **names[1]** and so on. This is depicted in Figure 9.4.

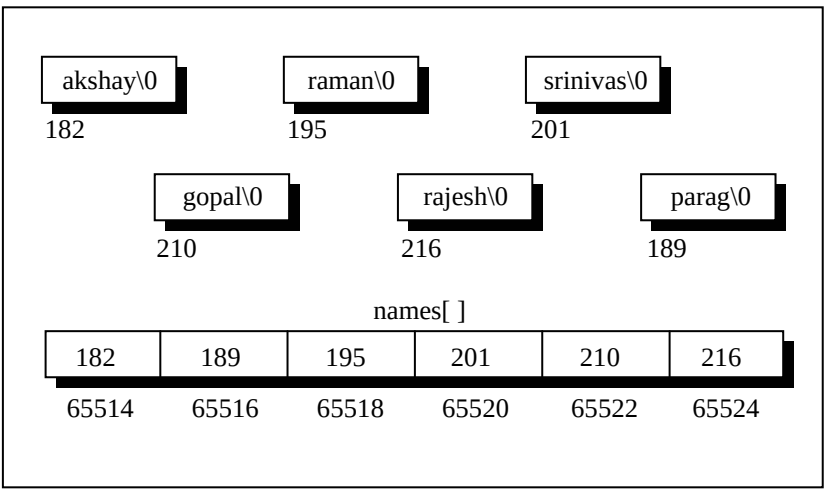


Figure 9.4

In the two-dimensional array of characters, the strings occupied 60 bytes. As against this, in array of pointers, the strings occupy only 41 bytes—a net saving of 19 bytes. A substantial saving, you would agree. But realize that actually 19 bytes are not saved, since 12 bytes are sacrificed for storing the addresses in the array **names[ ]**. Thus, one reason to store strings in an array of pointers is to make a more efficient use of available memory.

Another reason to use an array of pointers to store strings is to obtain greater ease in manipulation of the strings. This is shown by the following programs. The first one uses a two-dimensional array of characters to store the names, whereas the second uses an array of pointers to strings. The purpose of both the programs is very simple. We want to exchange the position of the names "raman" and "srinivas".

```
/* Exchange names using 2-D array of characters */
main( )
{
      char  names[ ][10] = {
```

```
                                    "akshay",
                                    "parag",
                                    "raman",
                                    "srinivas",
                                    "gopal",
                                    "rajesh"
                                } ;
    int  i ;
    char  t ;

    printf ( "\nOriginal: %s %s", &names[2][0], &names[3][0] ) ;

    for ( i = 0 ; i <= 9 ; i++ )
    {
        t = names[2][i] ;
        names[2][i] = names[3][i] ;
        names[3][i] = t ;
    }

    printf ( "\nNew: %s %s", &names[2][0], &names[3][0] ) ;
}
```

And here is the output...

```
Original: raman srinivas
New: srinivas raman
```

Note that in this program to exchange the names we are required to exchange corresponding characters of the two names. In effect, 10 exchanges are needed to interchange two names.

Let us see, if the number of exchanges can be reduced by using an array of pointers to strings. Here is the program...

```
main( )
{
    char  *names[ ] = {
```

```
                                "akshay",
                                "parag",
                                "raman",
                                "srinivas",
                                "gopal",
                                "rajesh"
                            } ;
    char  *temp ;

    printf ( "Original: %s %s", names[2], names[3] ) ;

    temp = names[2] ;
    names[2] = names[3] ;
    names[3] = temp ;

    printf ( "\nNew: %s %s", names[2], names[3] ) ;
}
```

And here is the output...

```
Original: raman srinivas
New: srinivas raman
```

The output is same as the earlier program. In this program all that we are required to do is exchange the addresses (of the names) stored in the array of pointers, rather than the names themselves. Thus, by effecting just one exchange we are able to interchange names. This makes handling strings very convenient.

Thus, from the point of view of efficient memory usage and ease of programming, an array of pointers to strings definitely scores over a two-dimensional character array. That is why, even though in principle strings can be stored and handled through a two-dimensional array of characters, in actual practice it is the array of pointers to strings, which is more commonly used.

Limitation of Array of Pointers to Strings

When we are using a two-dimensional array of characters we are at liberty to either initialize the strings where we are declaring the array, or receive the strings using **scanf( )** function. However, when we are using an array of pointers to strings we can initialize the strings at the place where we are declaring the array, but we cannot receive the strings from keyboard using **scanf( )**. Thus, the following program would never work out.

```
main( )
{
    char  *names[6] ;
    int  i ;

    for ( i = 0 ; i <= 5 ; i++ )
    {
        printf ( "\nEnter name " ) ;
        scanf ( "%s", names[i] ) ;
    }
}
```

The program doesn't work because; when we are declaring the array it is containing garbage values. And it would be definitely wrong to send these garbage values to **scanf( )** as the addresses where it should keep the strings received from the keyboard.

Solution

If we are bent upon receiving the strings from keyboard using **scanf( )** and then storing their addresses in an array of pointers to strings we can do it in a slightly round about manner as shown below.

```
#include "alloc.h"
main( )
```

```
{
    char  *names[6] ;
    char n[50] ;
    int  len, i ;
    char *p ;

    for ( i = 0 ; i <= 5 ; i++ )
    {
        printf ( "\nEnter name " ) ;
        scanf ( "%s", n ) ;
        len = strlen ( n ) ;
        p = malloc ( len + 1 ) ;
        strcpy ( p, n ) ;
        names[i] = p ;
    }

    for ( i = 0 ; i <= 5 ; i++ )
        printf ( "\n%s", names[i] ) ;
}
```

Here we have first received a name using **scanf( )** in a string **n[ ]**. Then we have found out its length using **strlen( )** and allocated space for making a copy of this name. This memory allocation has been done using a standard library function called **malloc( )**. This function requires the number of bytes to be allocated and returns the base address of the chunk of memory that it allocates. The address returned by this function is always of the type **void ***. Hence it has been converted into **char *** using a feature called typecasting. Typecasting is discussed in detail in Chapter 15. The prototype of this function has been declared in the file 'alloc.h'. Hence we have **#includ**ed this file.

But why did we not use array to allocate memory? This is because with arrays we have to commit to the size of the array at the time of writing the program. Moreover, there is no way to increase or decrease the array size during execution of the program. In other words, when we use arrays static memory allocation takes place.

Unlike this, using **malloc( )** we can allocate memory dynamically, during execution. The argument that we pass to **malloc( )** can be a variable whose value can change during execution.

Once we have allocated the memory using **malloc( )** we have copied the name received through the keyboard into this allocated space and finally stored the address of the allocated chunk in the appropriate element of **names[ ]**, the array of pointers to strings.

This solution suffers in performance because we need to allocate memory and then do the copying of string for each name received through the keyboard.

Summary

(a) A string is nothing but an array of characters terminated by '\0'.
(b) Being an array, all the characters of a string are stored in contiguous memory locations.
(c) Though **scanf( )** can be used to receive multi-word strings, **gets( )** can do the same job in a cleaner way.
(d) Both **printf( )** and **puts( )** can handle multi-word strings.
(e) Strings can be operated upon using several standard library functions like **strlen( )**, **strcpy( )**, **strcat( )** and **strcmp( )** which can manipulate strings. More importantly we imitated some of these functions to learn how these standard library functions are written.
(f) Though in principle a 2-D array can be used to handle several strings, in practice an array of pointers to strings is preferred since it takes less space and is efficient in processing strings.
(g) **malloc( )** function can be used to allocate space in memory on the fly during execution of the program.

Exercise

Simple strings